Sivas Cumhuriyet Üniversitesi

Ders Adı: Mobil Programlama I

Akademisyen: Doç. Dr. Mesut Polatgil

Takım İsmi: Toys Code

- Ali Akbaş - 2024481058
- Emrullah Altay - 2024481005
- Emirhan Yazıcı - 2024481265



Gemini Notebook

Modern, Kırılmaz Dart Kalesini İnşa Etmek (Sunum Ajandası)

Toys Code ekibi olarak amacımız, Dart’ın karmaşık gibi görünen ileri düzey mimarisini temel yapı taşlarına ayırmaktır. Bu sunum, resmi Dart dokümantasyonunun akademik bir sentezidir.



1. Temel Tesisat (Eşzamanlılık)

Olay Döngüsü (Event Loop) ve asenkron veri akış mekanizmaları.

2. Mimari Ölçekleme (Isolate)

Paylaşımsız bellek modeli, Thread'lere karşı Isolate mimarisi ve port iletişimleri.

3. Yapısal Güçlendirme (Null Güvenliği)

Milyar Dolarlık Hata'nın çözümü, Tip Evreni ve Derleyici Akış Analizi.

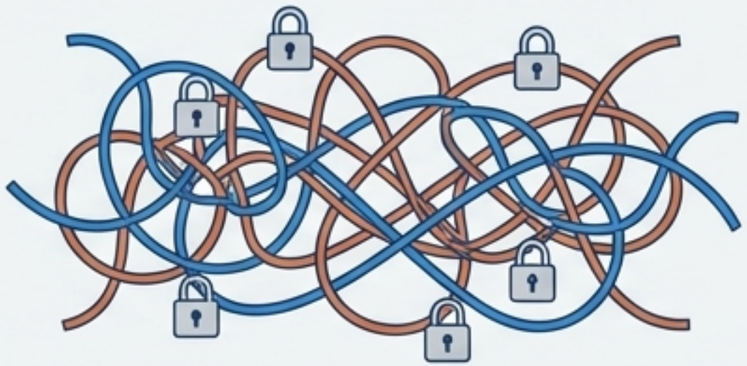
4. Yönetim ve Sentez

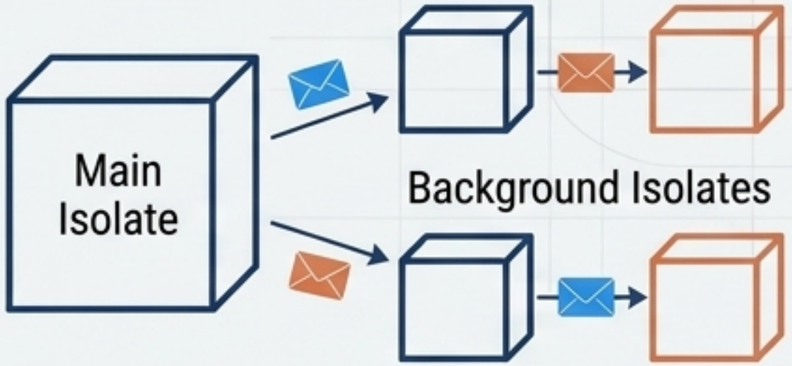
Dil sürümleme kuralları, Sentaks yapıları ve Mükemmel Uygulama Sentezi.

Gemini Notebook

Dart Mimarisine Genel Bakış: Eşzamanlılık (Concurrency) Felsefesi

Neden Geleneksel İş Parçacıkları (**Threads**) Yerine **Isolate**?





Geleneksel Model - Paylaşımlı Bellek

- Çoklu iş parçacıkları aynı bellek alanını (Memory Heap) paylaşır.
- **Risk:** Veri yarışları (Data Race) ve kilitlenmeler (Deadlocks).
- **Çözüm:** Karmaşık Mutex ve kilit (Lock) mekanizmaları gerektirir.

Dart'ın Aktör Modeli - Paylaşımsız Bellek

- Tüm Dart kodları izole edilmiş alanlarda (Isolate) çalışır.
- Varsayılan olarak tek bir Main Isolate vardır; UI ve temel işlemler burada akar.
- **Avantaj:** Her Isolate'ın kendine ait belleği ve olay döngüsü vardır. Durum (state) paylaşılmaz, sadece mesaj geçişi ile iletişim kurulur. Veri yarışı riski tasarımsal olarak sıfırlanmıştır.

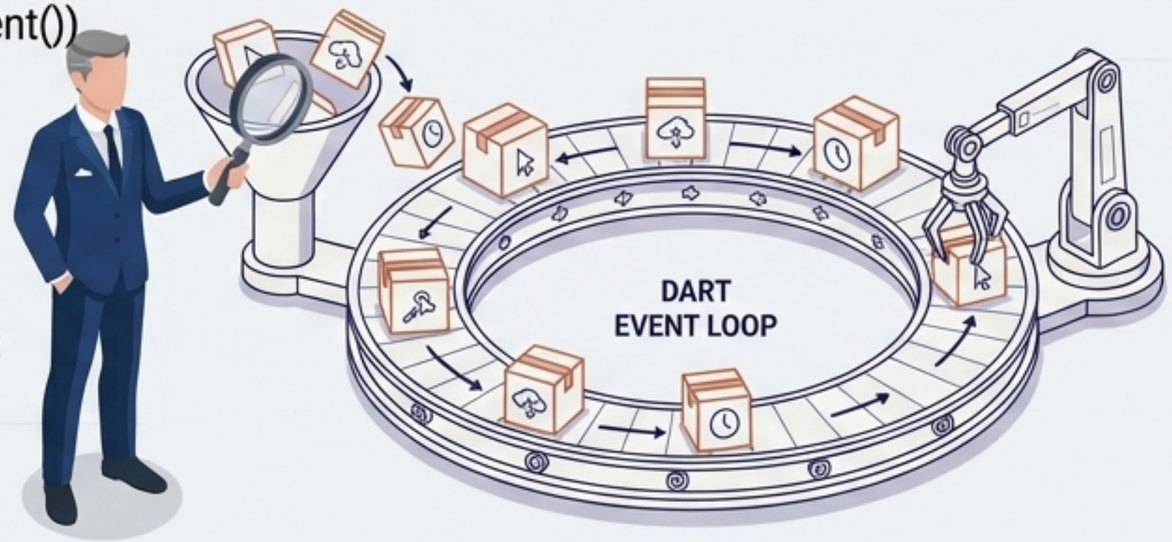


Gemini Notebook

Uygulamanın Kalp Atışı: Olay Döngüsü (Event Loop) Mekanizması

Asla Bloke Olmayan Tek İş Parçacığı (Single-Threaded, Non-Blocking)

Çalışma Mantığı: Dart çalışma zamanı modeli, bir while(eventQueue.waitForEvent()) döngüsü üzerine kuruludur.



FIFO Prensibi: Döngü, olayları kuyruğa girdikleri sıraya göre (İlk Giren İlk Çıkar) tek tek alır ve işler.

Olay Kuyruğu (Event Queue): Kullanıcı Kullanıcı dokunuşları, UI yeniden çizim istekleri, istekleri I/O ağ istekleri bu kuyruğa eklenir.

Mikrogörev Kuyruğu (Microtask Queue) Ayrıntısı:

Olay kuyruğundan daha yüksek önceliğe sahiptir. Dart, ana olaylara geçmeden önce mutlaka Mikrogörevleri (örn. Future.value) bitirir. Bu hiyerarşi uygulamanın akıcı kalmasını sağlar.

Gemini Notebook

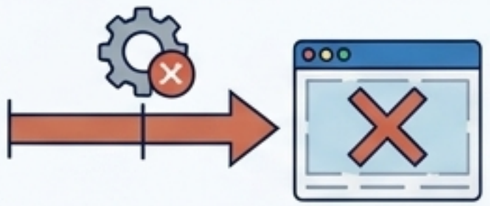
Asenkron Programlama Temelleri: Future ve async/await

CPU'yu Olay Döngüsüne Geri Veren Yapı

Code vs Execution

1 Senkron Blokaj: readAsStringSync() ⚠

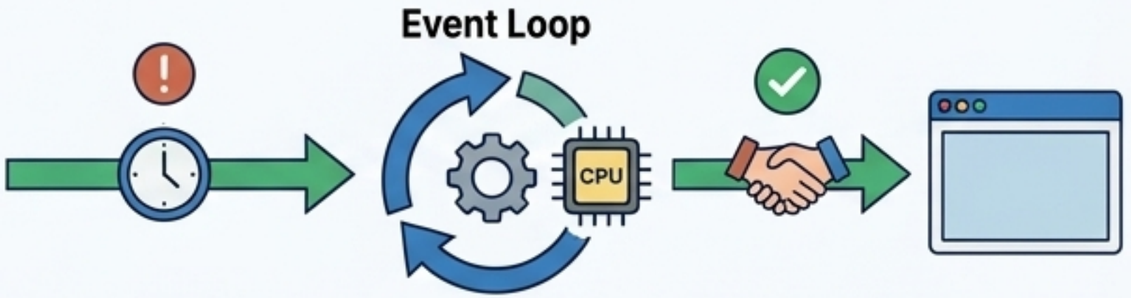
Sonuç: Dosya okunana kadar ana iş parçacığı donar. UI takılır (Jank), diğer olaylar işlenemez.



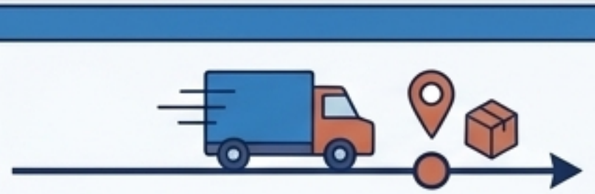
2 Asenkron Devir: await readAsString() ✓

Mekanizma: Fonksiyon **async** ile işaretlenir. **await** anahtar kelimesi işlemi başlatır, ancak sonucu beklerken yürütmeyi bloke etmeden duraklatır.

Sonuç: CPU kullanımı anında Olay Döngüsü'ne (Event Loop) iade edilir. UI akıcı kalır. İşlem bitince Future<T> sözü yerine getirilir ve kod kaldığı yerden devam eder.







FUTURE

Tek Seferlik İşlem

`async` `await` `then()`

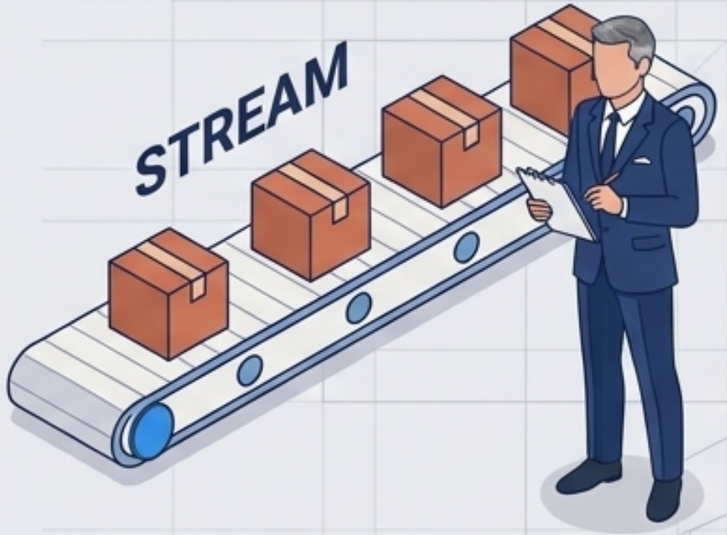
- Kullanım: HTTP ağ isteği,
- Dosya okuma.

Kural: await kullanımı, sadece gövdesi **async** ile işaretlenmiş fonksiyonlarda geçerlidir. İşlem sonucu bir nesne (Future) vaadi olarak döner.

Gemini Notebook

Reaktif Veri Akışları: Stream Mimarisi

Zaman İçinde Sağlanan Değerler Dizisi (Stream<T>)



Stream Nedir?

Gelecekte, zaman içine yayılmış olarak birden fazla değer (veya hata) yayma sözü veren asenkron veri borularıdır (örneğin WebSockets, UI dinleyicileri).

Asenkron Döngüler (await for)

Standart bir for döngüsünün asenkron versiyonudur. Stream yeni bir değer yayana kadar bekler., değeri alır, işler ve stream kapanana kadar döngü devam eder.

Veri Üretmek (yield ve async*)

Değer akışları döndüren jeneratör fonksiyonlar **async*** ile işaretlenir. **return** yerine **yield** kullanılır. **yield** fonksiyonu sonlandırmaz, stream'e bir damla veri ekler ve üretimi sürdürür.

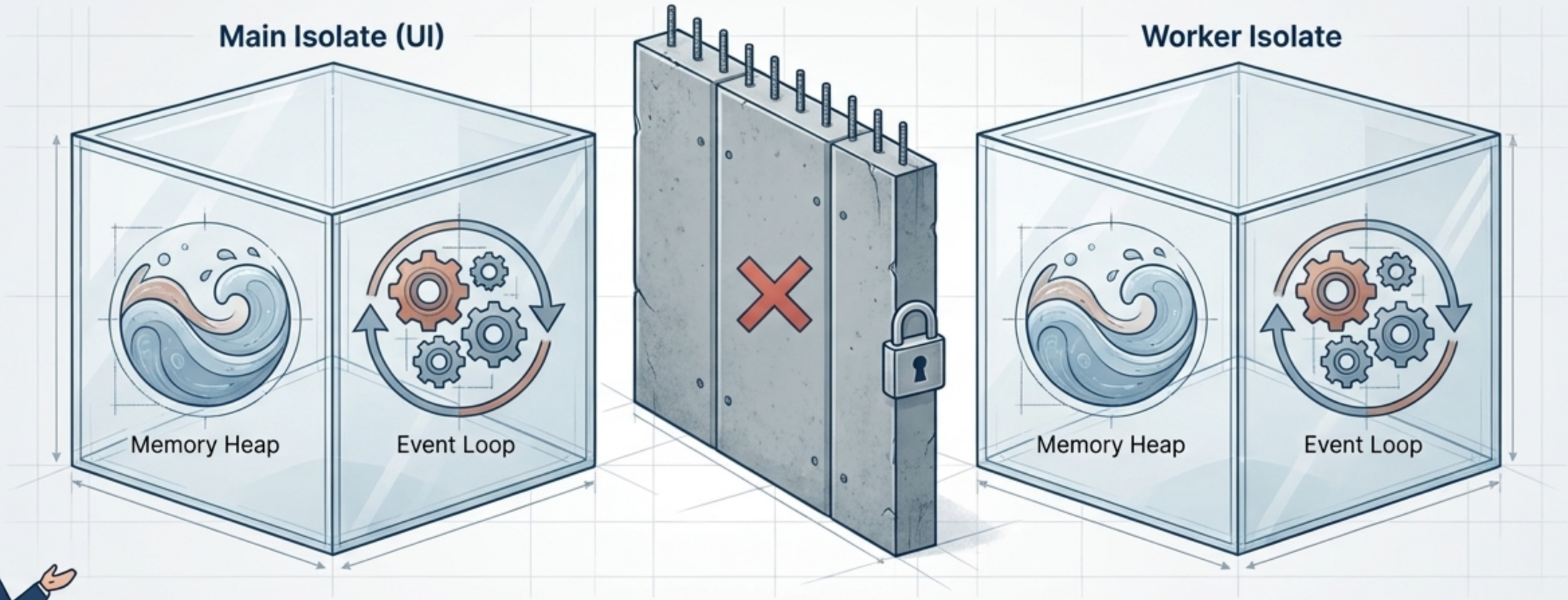
Yayın Tipleri

Tek Abonelikli (Single-subscription - dosya okuma) ve **Çoklu Yayın Yayın** (Broadcast - fare tıklamaları) olmak üzere iki strateji barındırır.

Gemini Notebook

Mimari Ölçekleme: Isolate ve Paylaşımsız Bellek Modeli

Aktör Modeline (Actor Model) Geçiş





Neden Isolate?

Çok büyük JSON dosyalarının ayrıştırılması, video/görüntü işleme gibi aşırı ağır CPU görevleri asenkron bile olsa Main Isolate'ın Event Loop'unu tıkar (Frame Drop/Donma yaşanır).

Çözüm: Arka Plan Çalışanları

Dart, bu yükü tamamen ayrı çekirdeklere devretmek için Isolate'ları kullanır.

Temel Felsefe (Shared-Nothing)

Her Isolate iş parçacıklarına benzer ancak asla durum (state) paylaşmaz. Kendi yalıtılmış belleği (Isolated Memory) vardır. Ana Isolate'daki global bir değişken, Worker Isolate'ta tamamen farklı bir kopyadır.

Gemini Notebook

Matris Karşılaştırması: Thread ve Isolate Paradigması

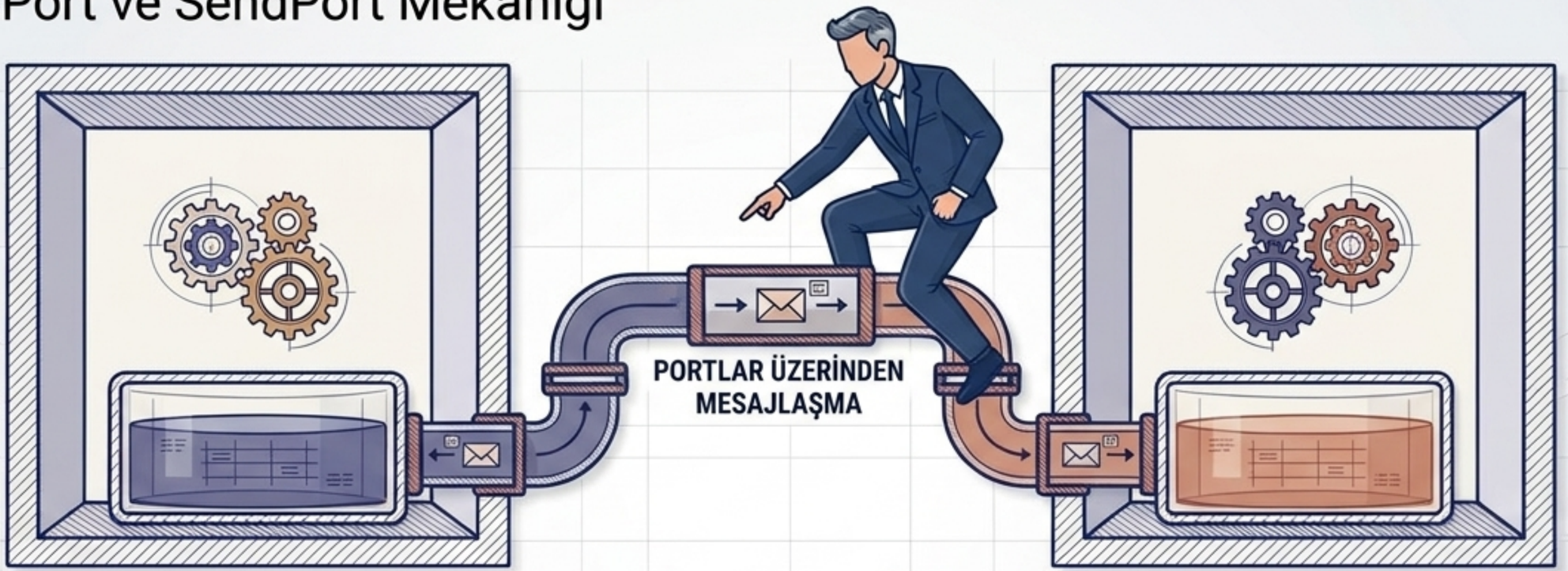
Kriter (Dimension)	Geleneksel İş Parçacığı (Threads)	Dart Isolate Mimarisi
Bellek Modeli (Memory)	Paylaşımlı Bellek (Shared Heap). Aynı state'e erişim.	İzole Bellek (Isolated Heap). Paylaşımsız model (Shared-Nothing).
Senkronizasyon	Mutex, Semaphore ve Lock mekanizmaları zorunludur.	Kilit (Lock) gerekmez. İletişim tamamen bağımsızdır.
Kritik Hata Riski	Veri Yarışı (Data Race) riski son derece yüksektir.	Tasarımsal olarak Veri Yarışı (Data Race) imkansızdır.
İletişim Yöntemi	Değişkenleri doğrudan okuma/yazma.	Portlar üzerinden kopyalanarak veya aktarılarak mesajlaşma.
Kullanım Maliyeti	Düşük bellek maliyeti, çok yüksek kod karmaşıklığı.	Nispeten yüksek bellek maliyeti, ancak kırılmaz ve güvenli kod.



Gemini Notebook

Isolate İletişimi: Portlar ve Stratejiler

ReceivePort ve SendPort Mekaniği



Çift Yönlü İletişim Ağı

Isolate'lar birbirlerinin belleğine bakamadığı için, iletişim yalnızca portlar üzerinden mesaj geçişi ile sağlanır.

Portların Rolü:

- **ReceivePort:** Diğer isolate'lardan gelen mesajları dinleyen ve işleyen alıcı istasyon (Bir Stream gibi çalışır).
- **SendPort:** Alıcı istasyona mesaj fırlatan verici. Her ReceivePort kendi SendPort'unu otomatik üretir.

Isolate Stratejileri

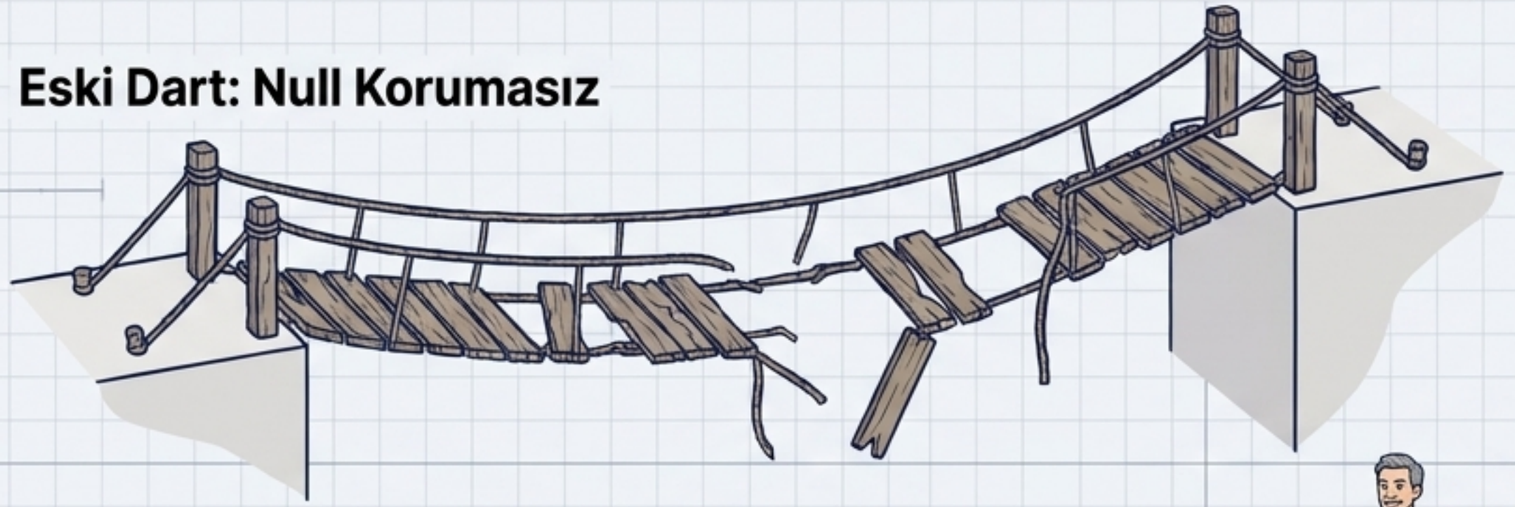
1. **Kısa Ömürlü Görevler (Isolate.run):** Tek bir ağır hesaplamayı (örn. jsonDecode) arka plana atar, sonucu bir Future olarak ana Isolate'a fırlatır ve kendini anında imha ederek belleği boşaltır. (Modern, düşük maliyetli yöntem).
2. **Uzun Ömürlü Görevler (Isolate.spawn):** Sürekli veri alışverişi gerektiren durumlar için kalıcı bir iletişim borusu (Handshake) kurar. Performans için Isolate Grupları mekanizmasını kullanır.

Gemini Notebook

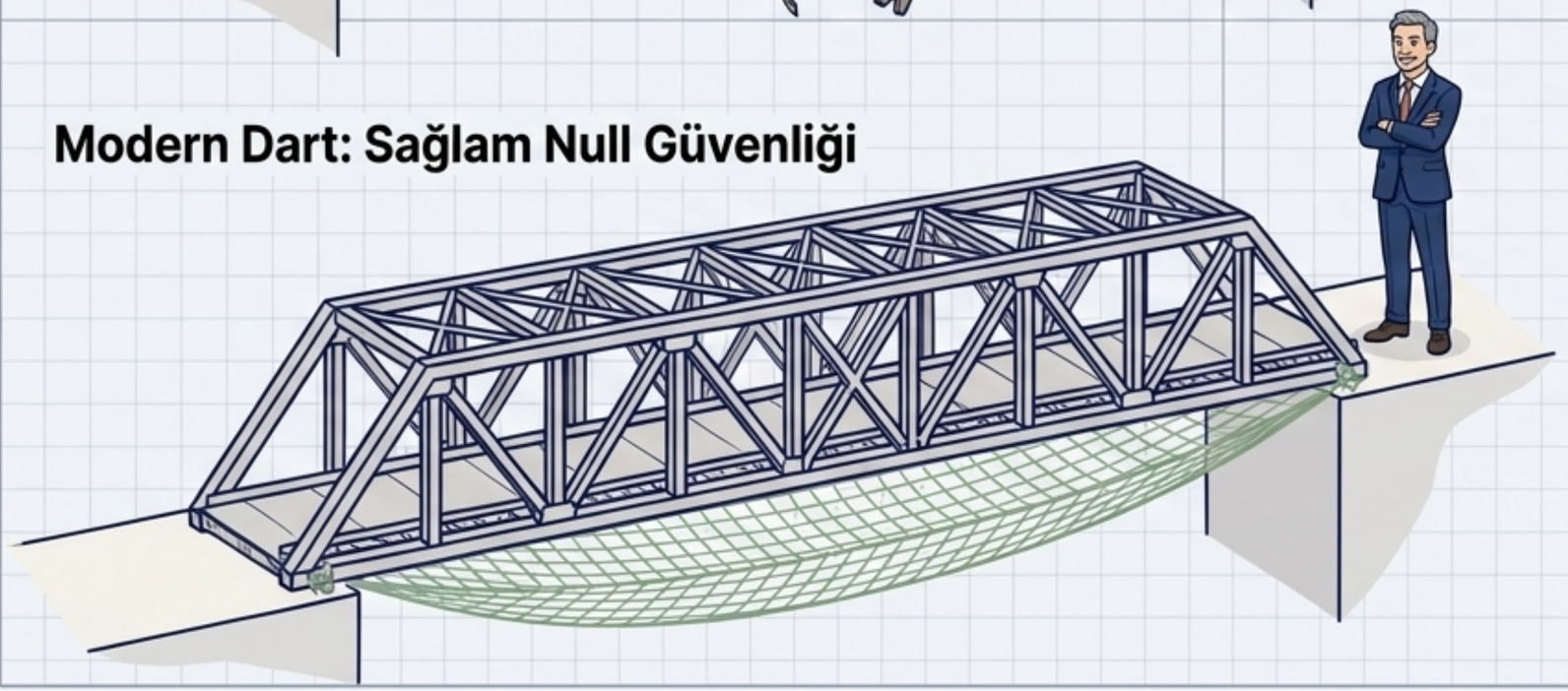
Yapısal Güçlendirme: Sağlam Null Güvenliği (Sound Null Safety)

Milyar Dolarlık Hatayı (Billion Dollar Mistake) Yok Etmek

Eski Dart: Null Korumasız



Modern Dart: Sağlam Null Güvenliği



Card 1

Problem: Milyar Dolarlık Hata

Eski sistemlerde her değişken gizlice null olabilirdi. Çalışma zamanında (**runtime**) **null** bir nesnede **.length** çağırmak uygulamanın anında çökmesine (NoSuchMethodError) sebep olurdu.

Card 2

Çözüm: Varsayılan Olarak Null Olamaz

Dart'ın devrimi. Siz tür bildirimine özel olarak **?** (örn. String?) eklemediğiniz sürece hiçbir değişken null değerini alamaz.

Card 3

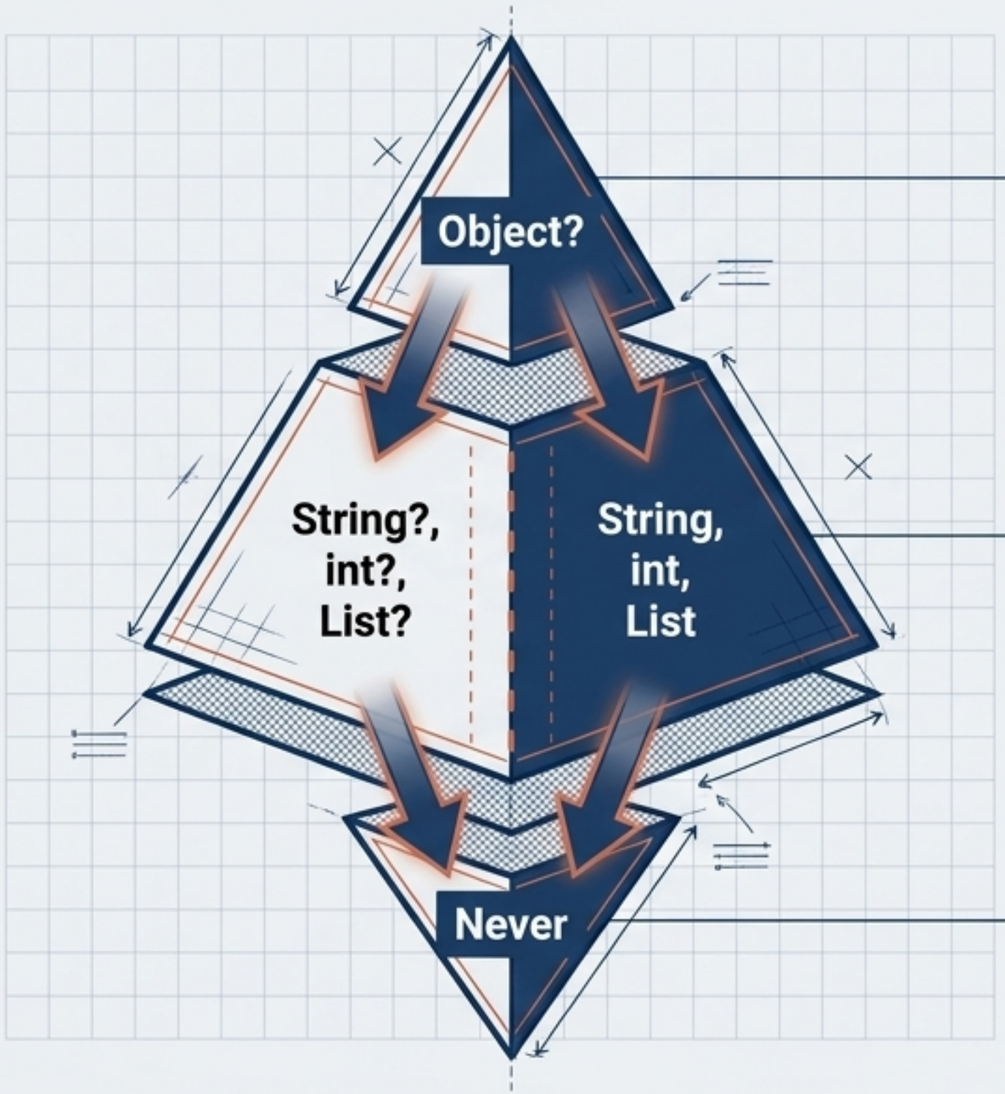
Tamamen Sağlam (Soundness) Garantisi

Tip sistemi bir değişkenin null olamayacağını belirlerse, bu derleyici tarafından garanti altına alınır. Sadece çökmeler önlenmez; derleyici gereksiz null kontrollerini silerek çok daha hızlı makine kodu üretir.

Gemini Notebook

Dart Tip Evreninin Anatomisi (The Type Universe)

Non-Nullable ve Nullable Tiplerin Hiyerarşisi



Zirve Tip (Top Type - Object?)

Dart'taki her şeyin (null dahil) en üst atasıdır. Herhangi bir değerin gelmesine izin vermek istiyorsanız **Object** yerine **Object?** kullanmalısınız.

Gövde Tipler (Nullable ve Non-Nullable Yarılar)

Tip evreni ortadan ikiye bölünmüştür. **String?** gibi nullable tipler **null**'a izin verir ama metodlarına doğrudan erişimi yasaklar. **String** gibi non-nullable tipler güvenle metod çağrısına izin verir. Implicit downcast (örtük aşağı dönüşüm) kesinlikle yasaktır, manuel cast (**as**) gerektirir.

Taban Tip (Bottom Type - Never)

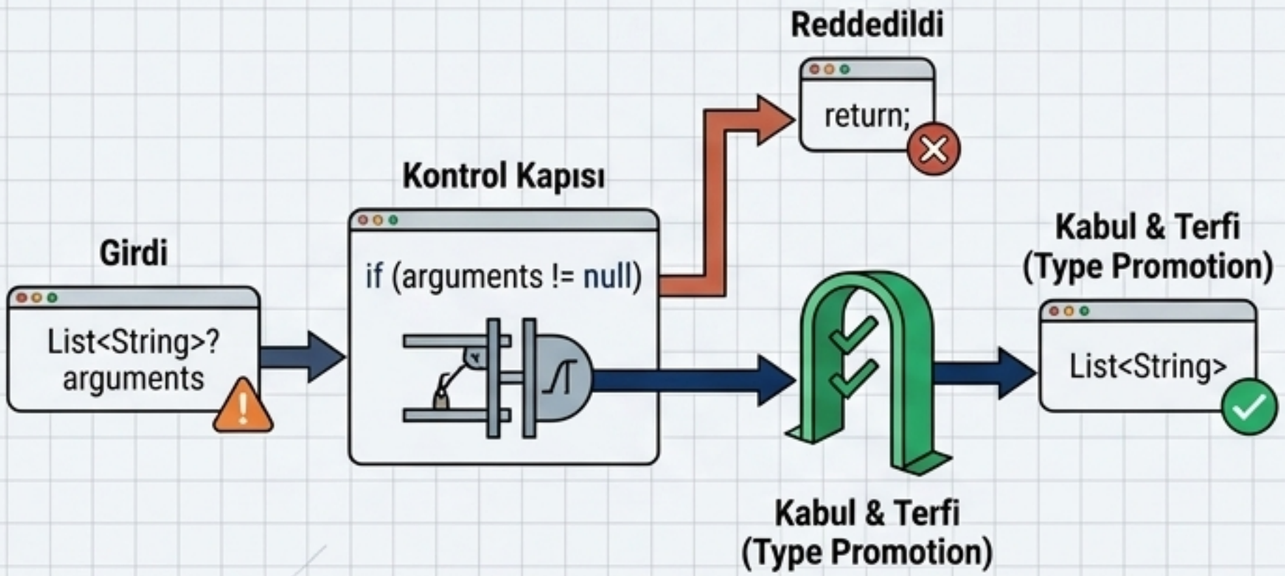
Hiçbir değere sahip olmayan uç tiptir. Kodun hiçbir zaman başarıyla tamamlanmayacağını, bir istisna (throw) fırlatacağını veya sonsuz döngüye gireceğini derleyiciye ispatlamak için kullanılır.



Gemini Notebook

Derleyici Akış Analizi (Flow Analysis)

Kodun Statik Doğruluğunun İspatı ve Tip Terfisi (Type Promotion)



Derleyici Bir Dedektiftir

Dart analizcisi, if/else bloklarını, return ve throw ifadelerini adım adım takip eder.

Erişilebilirlik Analizi (Reachability)

Kodun belirli bir satırına ulaşılıp ulaşılamayacağını hesaplar. Bir fonksiyonda erken return varsa, alt satırlardaki riskler yoksayılır.

Kesin Atama Analizi (Definite Assignment)

Null olmayan (non-nullable) yerel bir değişkenin, kodun her olası yolunda okunmadan önce başlatıldığını kesin olarak doğrular. Aksi halde derlemeyi durdurur.

Otomatik Tip Yükseltme (Type Promotion)

if (arguments != null) gibi bir güvenlik kapısı kullanıldığında, derleyici o blok içerisinde List<String>? tipini otomatik olarak güvenli List<String> tipine terfi ettirir. Artık güvenle işlem yapılabilir.



Gemini Notebook

Sentaks ve Yönetim: Anahtar Kelimeler ve Operatörler

Null Evreninde Güvenle Gezinme Araçları

KONTROL AKIŞI

if	else	switch	case	break	continue
await	try	catch	finally	throw	rethrow

ASENKRON VE ZAMAN

async	await	yield	sync*

DÜZENLEYİCİLER (MODIFIERS)

class	mixin	enum	extension	typedef
late	required	final	const	abstract
sealed	static	external		

Gecikmeli Başlatma (late)	
Kısıtlamaları derleme zamanında değil, çalışma zamanında uygula komutudur. Değişken non-nullable'dır ancak değeri daha sonra atanacaktır. Atanmadan okunursa anında istisna fırlatır.	
Zorunlu Parametre (required)	
Named (isimli) parametrelerde çağırıcının o değeri kesinlikle iletmesini zorunlu kılar.	
Null-Aware Operatörler Sözlüğü	
?.	**(Güvenli Çağrı):** Alıcı null ise zinciri anında koparır (kısa devre yapar) ve çökmeden null döner.
?[ ]	**(Güvenli İndeks):** Liste veya Map'te null kontrolü yaparak eleman arar.
..	**(Null-Aware Cascade):** Nesne null değilse ardışık işlemleri bağlar.
!	**(Null Non-Assertion - Bang):** Geliştiricinin derleyiciy "Bunun null olmadığını garanti ediyorum, as String'e dönüştür" emridir. Yanlışsa çalışma zamanında kodu patlatır.



Gemini Notebook

Dil Sürümleme ve Geriye Dönük Uyumluluk (Language Versioning)

Büyük Projelerde Kademeli Göç Yönetimi

Ekosistem Yönetimi

Dart, dilin birden fazla sürümünü aynı anda destekleme yeteneğine sahiptir. Modern özelliklerle yazılmış kodlar ile eski (legacy) kodlar aynı projede var olabilir.



Proje Bazlı Kısıtlama (pubspec.yaml)

Tüm projenin varsayılan SDK ortamı ve null-safety kuralları pubspec içindeki sdk: '>=2.17.0 <3.0.0' gibi kısıtlamalarla belirlenir.

Dosya Bazlı İzolasyon

Eski bir kütüphane henüz null-safety göçünü tamamlamadıysa, dosyanın en üstüne yazılan // @dart = 2.9 gibi dil sürümü etiketleriyle o dosya özel olarak eski kurallarla çalıştırılabilir. Bu özellik devasa projelerin aşama aşama güncellenmesini sağlar.

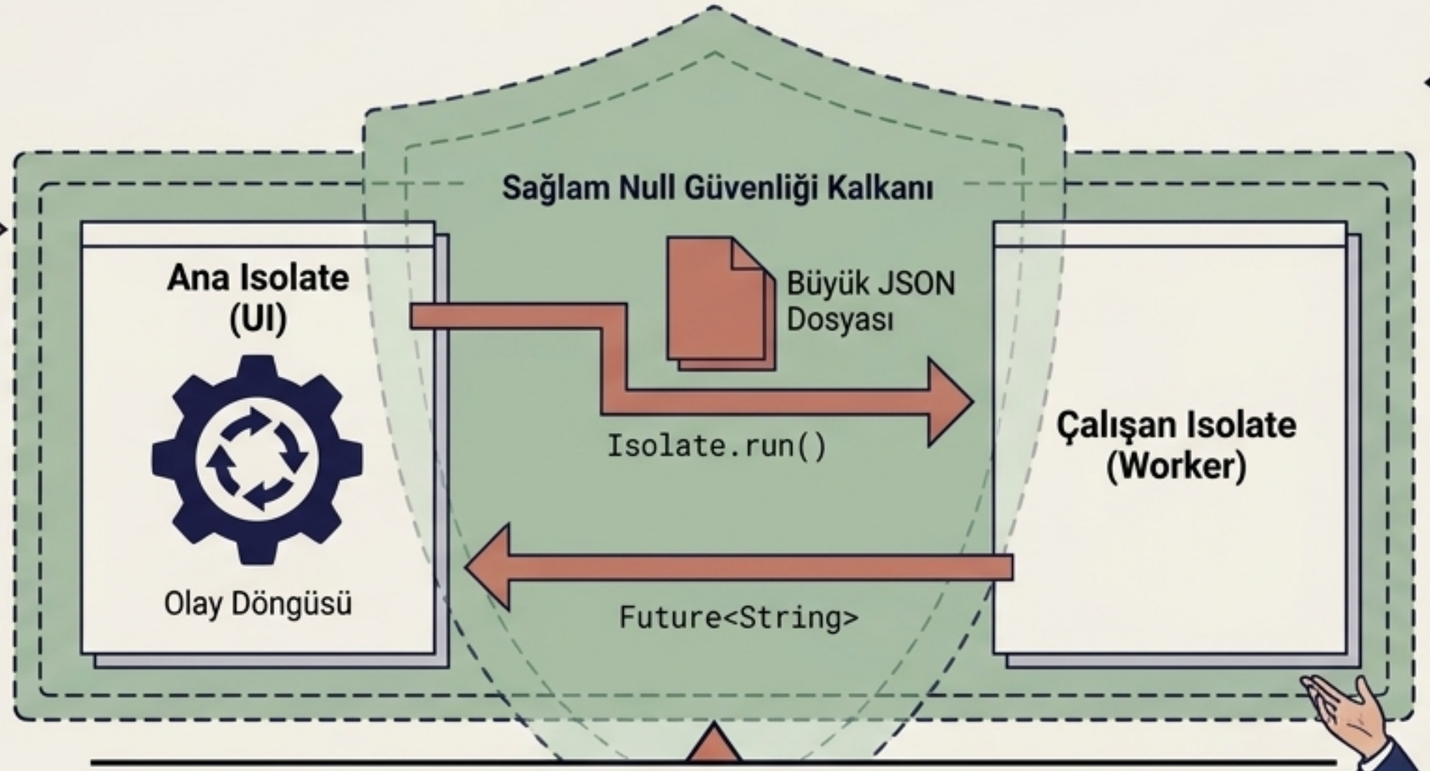
Gemini Notebook

Mimari Sentez: Modern Dart Uygulaması

Kırılmaz, Akıcı ve Ölçeklenebilir Yapının Taslağı

1. Kullanıcı Arayüzü (UI) ve Akıcılık

Uygulamanın kalbi olan **Olay Döngüsü** (Event Loop) ana iş parçacığında kesintisiz çalışır. **async/await** ile ağ istekleri sistemi bloke etmeden işlenir.



2. Ağır Yük Delegasyonu

UI'ı donduracak devasa JSON ayrıştırma işlemleri **Isolate.run()** üzerinden Çalışan Isolate'a (Worker) fırlatılır.
Veri, paylaşımsız bellek sayesinde **yarış riski (Data Race)** olmadan ana döngüye **Future** olarak geri döner.

3. Kalkan ve Sigorta

Gelen bu veriler, Dart 3'ün Sağlam Null Güvenliği (Sound Null Safety) çelik köprüsünden geçer. Akış Analizi devreye girerek çalışma zamanı çökmelerini (Billion Dollar Mistake) henüz kod yazılırken derleme zamanında (**compile-time**) imha eder.



Sonuç: Saniyede 60/120 kare (fps) hızında çalışan, asla donmayan ve asla boş (null) referans hatası vermeyen bir mühendislik harikası.

Gemini Notebook
Gemini Notebook

Ders Değerlendirmesi ve Kapanış

Mobil Programlama I İçin İleri Düzey Kazanımlar

Akademik Kazanım:

Dart'ın sadece bir sentaks olmadığı; Event Loop, Isolate ve Sound Null Safety yapıtaşları üzerine inşa edilmiş kusursuz bir mimari sistem olduğu kanıtlanmıştır.



Pratik Değer:

Bu mimari prensiplere hakimiyet, öğrencilere standart mobil uygulamaların ötesinde; profesyonel, performanslı ve çökmelere karşı bağışıklık kazanmış endüstri standardı yazılımlar üretme yetkinliği kazandırmaktadır.

Teşekkürler:

Toys Code takımı olarak, Dart mimarisinin derinliklerine yaptığımız bu akademik yolculukta bizi dinlediğiniz için teşekkür ederiz.

Gemini Notebook